

Student Management System (Spring Boot CRUD)

Project Overview

This is a **Spring Boot REST API project** that performs full CRUD operations for managing student data. It uses **Spring Boot, Spring Data JPA, and MySQL (or H2/MySQL depending on setup)**.

Tech Stack

- Java 17+
 - Spring Boot
 - Spring Web (REST APIs)
 - Spring Data JPA
 - Hibernate
 - MySQL (or H2 Database)
 - Maven
 - Postman (for API testing)
-

Project Architecture

Controller → Service → Repository → Database

Project Structure

```
com.example.studentapp
|
|— controller
|   └─ StudentController.java
|
|— service
|   └─ StudentService.java
|
|— repository
|   └─ StudentRepository.java
|
|— entity
|   └─ Student.java
|
└─ StudentappApplication.java
```

Entity (Student)

- Represents database table
- Annotated with @Entity

Fields:

- id (Primary Key)
 - name
 - email
-

Repository

public interface StudentRepository extends JpaRepository<Student, Long>

Features:

- save()
 - findAll()
 - findById()
 - deleteById()
-

Service Layer

Handles business logic:

Features implemented:

- Create student
 - Get all students
 - Get student by ID
 - Update student
 - Delete student
-

REST API Endpoints

1. Create Student

POST /students

Request Body:

```
{
  "name": "Manikandan",
  "email": "mani@gmail.com"
}
```

2. Get All Students

GET /students

3. Get Student by ID

GET /students/{id}

4. Update Student (PUT/PATCH)

PUT /students/{id}

PATCH /students/{id}

Request Body:

```
{  
  "name": "Updated Name",  
  "email": "updated@gmail.com"  
}
```

5. Delete Student

DELETE /students/{id}

How to Test in Postman

Step 1:

Select HTTP method (GET/POST/PUT/PATCH/DELETE)

Step 2:

Enter URL:

http://localhost:8080/students

Step 3:

For POST/PUT/PATCH:

- Go to Body → raw → JSON

Step 4:

Click **Send**

How to Run Project

Step 1: Start Spring Boot

mvn spring-boot:run

Step 2: Check server

http://localhost:8080

Database Configuration (if MySQL used)

spring.datasource.url=jdbc:mysql://localhost:3306/studentdb

spring.datasource.username=root

spring.datasource.password=yourpassword

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

Features Completed

- ✓ Create Student
 - ✓ Read All Students
 - ✓ Read Student by ID
 - ✓ Update Student
 - ✓ Delete Student
 - ✓ REST API integration
 - ✓ Database integration (JPA + Hibernate)
-

Key Learning Outcomes

- Spring Boot project setup
 - REST API development
 - CRUD operations
 - Layered architecture
 - Postman testing
 - JPA repository usage
-

Future Enhancements

- Exception Handling (@ControllerAdvice)
- DTO Layer
- Input Validation

- Swagger API Documentation
 - Pagination & Sorting
 - Security (JWT Authentication)
-

 **Author**

Manikandan

Spring Boot Developer (Learning Project)